

# A Cognitive and Emotional Multi-Agent System for Ludo Game Simulation

Kevin Nettikadan  
*nettikadankevin@yahoo.com*

September 18, 2026

## 1 Details of the proposal

The project consists of the design and implementation of a Multi-Agent System (MAS) to simulate a complete game of Ludo. The primary goal is to model different cognitive and emotional player profiles using a BDI (Belief-Desire-Intention) architecture, leveraging VEsNA Pro frameworks to govern agent decision-making based on static temperaments and dynamic moods.

### 1.1 Full specification of your proposal

The following section presents the original description submitted in the initial proposal form. This served as the conceptual foundation for the project, though the system was significantly expanded and refactored during the actual development phase to better reflect real-world dynamics.

*I will implement a Multi-Agent System playing the board game "Ludo" using the VEsNA pro framework, as suggested by the professor. The game involves 4 players, each controlling 4 tokens. The goal is to race all tokens from a starting base to a central home area based on the roll of a dice. A crucial mechanic is interaction: landing on a square occupied by an opponent's token "eats" it, sending it back to the start. I will implement 4 Agents representing the players. These agents must perceive the board state and autonomously decide which token to move when multiple options are available. The 4 agents have different personality features to create distinct strategies (e.g., an "Aggressive" profile that prioritizes capturing opponents versus a "Prudent" profile that prioritizes safety).*

The system evaluates how different predefined personalities interact within a competitive, multi-player environment with a single winner. The project integrates logical reasoning (Jason), environment management (CArtAgO), and emotional evaluation (VEsNA Pro). The finalized system is composed of:

- A Ludo game environment that manages the objective game state, strict rules, and player interactions.
- Multiple agent profiles: 3 Static Cognitive Agents (Aggressive, Prudent, Balanced) and 1 Dynamic Emotional Agent (Emotive).
- A decision-making framework allowing agents to adapt strategies based on their current mood and temperament.
- A custom user interface for observing the game progress and agent behaviors.

During the implementation, several architectural improvements and functional expansions were introduced compared to the initial plan:

- **Dummy Server:** Since the VEsNA framework was originally designed for embodied agents in 3D virtual environments (Godot engine), a lightweight Python WebSocket server was implemented to act as a dummy body endpoint, allowing the four agents to establish their required WebSocket connections without the need for a graphical 3D environment.
- **Enhanced Graphical Feedback:** Introduction of a multi-segment directional arrow system within the custom Java Swing GUI to visually trace agent decisions and trajectories before they are finalized on the board.
- **Concurrency and Pause State:** Implementation of a thread-safe Pause mechanism allowing the user to freeze the CArtAgO artifact and inspect Jason reasoning logs in real-time without causing deadlocks between the system and the interface.

## 1.2 The kind of your proposal

This is a creative project classified as hard. It requires integrating the VEsNA Pro emotional engine with standard BDI logic, mapping discrete board positions to complex tactical evaluations (e.g., computing circular modular distances to hunt opponents without overshooting), and carefully calibrating VEsNA temper arrays to ensure distinct and recognizable behaviors across all four agents.

## 2 Domain Model

This section describes the system’s domain model, defining the rules, the physical entities, and the spatial logic of the simulated Ludo environment.

### 2.1 The Ludo Board and Game Rules

Ludo is a classic board game for 2 to 4 players derived from the ancient Indian game Pachisi. It was chosen as the simulation environment because its mechanics naturally produce the kinds of multi-agent interaction-cooperation, competition, risk management, and emotional reactions, that the VEsNA Pro framework is designed to model.

In this simulated environment, the game involves exactly 4 players (represented by our autonomous BDI agents), each controlling 4 tokens. All tokens begin the game off the board in a dedicated starting area called the **Base**.

The turn structure is strictly sequential. On each turn, the active agent rolls a single six-sided dice (values 1 to 6). A token may only leave the Base and enter the shared outer track upon rolling a **6**, which represents the exclusive condition for deployment. Once in play, tokens advance clockwise around the outer perimeter by exactly the number of steps shown on the dice.

A crucial interaction mechanic governs the shared perimeter: if an agent’s token lands on a square already occupied by an opponent’s token, the opponent’s token is **captured** (colloquially referred to as being “eaten”) and immediately returned to its Base, where it must wait for another roll of 6 to re-enter the game. This mechanic is the primary driver of conflict and emotional reaction within the simulation.

While the shared outer track exposes tokens to these attacks, each player has a private colored **safe zone** of 6 squares leading toward the central area, accessible only to tokens of that specific color. Tokens navigating this safe zone are completely immune to capture. The first agent to successfully move all 4 tokens into the central **Home** area wins the simulation.

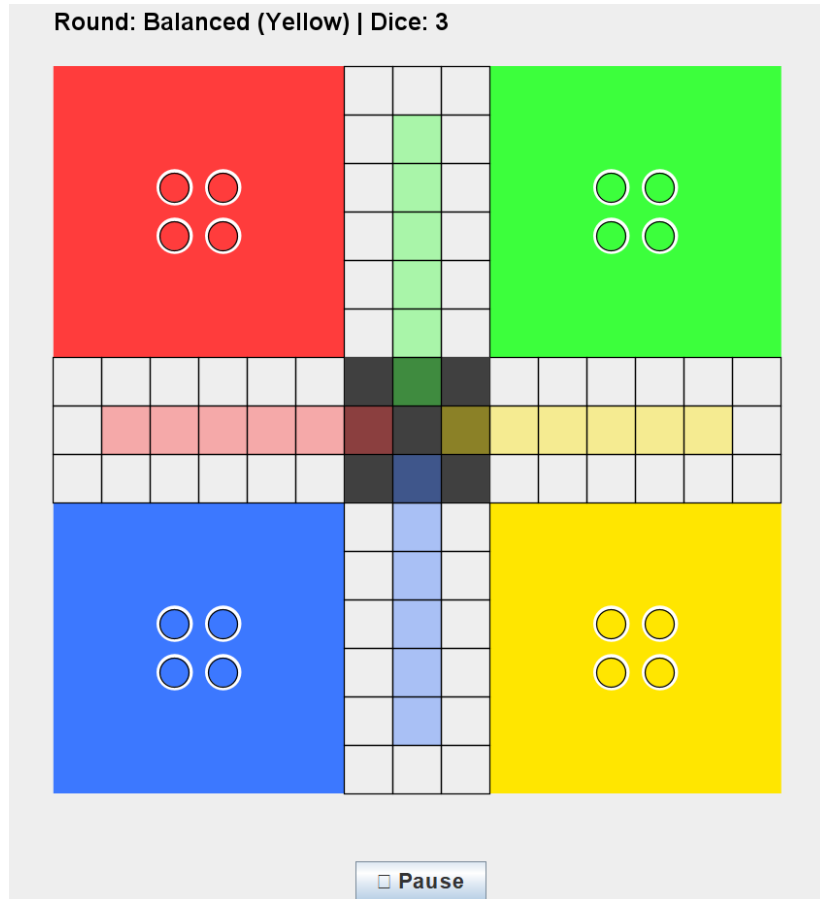


Figure 1: Initial board configuration in the Java Swing GUI. All 16 tokens are in their colored Base areas (corners). The shared white perimeter (52 squares), the four colored safe-zone corridors, and the central dark Home area are clearly visible. The GUI also displays the current player and the last dice roll in the status bar at the top.

## 2.2 The Board and Spatial Logic

To easily manage the board state without hardcoding every single path for each player, the main perimeter of the Ludo board is mathematically modeled as a **cyclical array of 52 absolute squares**. Each player’s journey starts at a fixed offset within this array. Since 4 players are equidistant on a 52-square board, their starting positions are separated by exactly 13 squares each.

To calculate overlaps and potential attacks across this shared cyclical structure, agents use modular arithmetic. The absolute position on the shared board is calculated using the following formula:

$$AbsPos = (PlayerID \times 13 + LogicalPos) \pmod{52}$$

where `PlayerID` ranges from 0 to 3 and `LogicalPos` represents the token’s position relative to its own starting point. This mathematical abstraction allows any agent to compare its absolute position against any opponent’s absolute position, predicting collisions regardless of their distinct starting points on the physical board.

Beyond the shared perimeter, each player has a private **safe zone** consisting of 6 squares (logical indexes 51 to 56) that leads to the final **Home** square (index 57). Tokens in this zone are only accessible to the owning player and are immune to any form of capture.

## 2.3 Token States

Each agent controls 4 tokens. To simplify the BDI reasoning process, a token can exist in one of four distinct macro-states during the simulation, tracked by an integer representing its logical position:

- **Base (-1):** The token is off the board. It requires a specific triggering event (a dice roll of exactly 6) to be deployed into the active arena at position 0.
- **In Play (0 to 50):** The token is navigating the outer shared perimeter. In this state, it is exposed to attacks and can itself attack opponents landing on the same absolute square.
- **Safe Zone (51 to 56):** The token has entered the final straight. It is completely invulnerable to attacks and is heading toward the winning condition.
- **Home (57):** The token has reached the central winning area. It no longer participates in the game. An agent wins when all 4 of its tokens reach this state.

### 3 Agents and Artifacts

This section details the roles, psychological profiles and programmed behaviors of the specific components designed for the simulation. The system consists of one shared CArtAgO Artifact and four distinct Jason/VEsNA Pro agents.

#### 3.1 LudoBoard Artifact

The LudoBoard Artifact, developed in Java using the CArtAgO framework, acts as the shared physical environment and the absolute source of truth for the game state. It is responsible for:

- Maintaining `positions` [4] [4], tracking the logical position of all 16 tokens (4 agents  $\times$  4 tokens).
- Generating dice rolls via the `@OPERATION rollDice()` method.
- Resolving spatial conflicts by detecting overlaps using the absolute position formula and sending captured tokens back to Base.
- Exposing **observable properties** `current_player`, `dice`, `board_state`, and individual properties of `token(P, T, Pos)`, that the Jason agents continuously monitor to trigger their BDI reasoning cycles.
- Rendering the Java Swing GUI and managing concurrent access between the JaCaMo execution thread and the Swing Event Dispatch Thread (EDT).

The system follows the standard JaCaMo three-layer architecture: the **Jason BDI engine** governs agent reasoning in `.asl` files, the **CArtAgO framework** manages the shared LudoBoard artifact, and **VEsNA Pro** adds the temperament-based plan selection layer on top of standard Jason deliberation. Agents perceive the board exclusively through CArtAgO observable properties and act on it through artifact operations (`rollDice`, `moveToken`), never communicating directly with each other.

#### 3.2 Temper Configuration Overview

Each agent is assigned a `temper` vector in the `.jcm` configuration file. This vector defines the agent’s psychological baseline across three static personality dimensions (`aggression`, `safety`, `advance`) and, for the fourth agent, two additional dynamic mood dimensions declared with the `[mood]` tag. Table 1 provides a comparative overview.

The Emotive agent adopts a neutral static baseline (0.5, 0.5, 0.5) equidistant from all personality extremes. The behavioral difference emerges exclusively from the dynamic `anger/joy` dimensions, which shift at runtime in response to game events.

| Agent             | aggression | safety | advance | anger | joy  |
|-------------------|------------|--------|---------|-------|------|
| Aggressive (Red)  | 0.9        | 0.1    | 0.5     | —     | —    |
| Prudent (Green)   | 0.1        | 0.9    | 0.5     | —     | —    |
| Balanced (Yellow) | 0.5        | 0.5    | 0.7     | —     | —    |
| Emotive (Blue)    | 0.5        | 0.5    | 0.5     | 0.1*  | 0.8* |

Table 1: VEsNA Pro temper configuration for each agent. Values marked with \* are mutable mood propensities ([mood]) that are altered at runtime through plan effects.

### 3.3 Aggressive Agent (Red)

This agent represents a predatory playstyle. Its VEsNA **temper** strongly prioritizes **aggression** (0.9) while minimizing **safety** (0.1). Its plan hierarchy is:

1. **Eat an opponent** (@eat\_enemy, aggression = 1.0): If any token can land on an occupied square, this plan is the highest priority.
2. **Deploy a new token** (@exit\_base, aggression = 0.8): On a roll of 6, the agent expands its presence on the board aggressively.
3. **Hunt the nearest prey** (@hunt\_preyl, aggression = 0.6): The agent seeks to capture the closest opponent. Using Jason’s .findall and .min internal actions, the agent computes the **circular modular distance** to every enemy token after each possible move. It selects the token that positions itself closest behind a target without overshooting, maximising the chance of a capture on the following turn.
4. **Enter safe zone** (@advance\_safe): A purely opportunistic retreat, applied only when the move enters into the protected corridor.
5. **Fallback**: Passes the turn if no valid move exists.

### 3.4 Prudent Agent (Green)

The Prudent agent is highly risk-averse. Its VEsNA priorities are mapped heavily toward the **safety** dimension (0.9). Its decision hierarchy strictly favors:

1. **Enter the safe zone** (@reach\_safety, safety = 1.0): The highest-priority plan. If any token is able to reach the private corridor in a single move, the agent will always take it.
2. **Advance an exposed token** (@advance\_token, safety = 0.8): Moving a token that is already on the shared perimeter is preferred over deploying new ones, as it reduces the number of exposed targets.
3. **Deploy a new token** (@exit\_base, safety = 0.2): The lowest priority. The agent only deploys a new token when it rolls a 6 and has no other legal action available.

4. **Fallback:** Passes the turn.

### 3.5 Balanced Agent (Yellow)

The Balanced agent simulates rational human play by dynamically evaluating the board state before committing to a strategy. Its key innovation is the **army strategy**: using Jason’s internal `.count()` function, it evaluates how many of its tokens are currently active on the shared perimeter (`active_tokens`). Its plan hierarchy is:

1. **Eat an opponent** (`@eat_enemy`, aggression = 0.7): Opportunistic captures are taken but with less urgency than the Aggressive agent.
2. **Enter the safe zone** (`@reach_safety`, safety = 0.8): Protecting a token by moving it into the corridor is a high priority.
3. **Deploy urgently** (`@exit_base_urgent`): If less than 2 tokens are in play, a roll of 6 is immediately used to deploy. This plan has a temper perfectly matching the agent’s baseline (0.5, 0.5, 0.5), giving it the highest possible similarity score in this scenario.
4. **Advance** (`@advance_token`): Standard progression once the army is sufficiently deployed.
5. **Fallback:** Passes the turn.

### 3.6 Emotive Agent (Blue)

The Emotive agent is the experimental subject for the VEsNA Pro dynamic mood engine. Its neutral static baseline (0.5, 0.5, 0.5) means that behavioral direction is driven almost entirely by the mutable `anger`/`joy` mood values.

**Calm state:** Initial mood (`anger` = 0.1, `joy` = 0.8) steers VEsNA Pro toward plans annotated with low anger and high joy: cautious advancement and calm deployment.

**Trauma trigger:** The agent maintains a working memory using the belief `in_play(T)`, added when a token moves onto the board. If `token(Me, T, -1)` fires and `in_play(T)` is true, the agent infers a capture event and triggers `!react_to_death(T)`. The `@got_eaten` plan raises `anger` significantly via VEsNA Pro effects.

**Angry state:** With high `anger`, the similarity calculation strongly favors plans annotated with `anger`(0.9), steering the agent toward aggressive deployment and pursuit.

**Catharsis:** The agent returns to a calmer behavioral state when it successfully captures an opponent via a specific retaliatory plan (`@vendetta`). The execution of this plan directly alters the mood variables, reducing `anger` and restoring `joy`, allowing the emotional loop to close. The technical implementation of this

transition, including the specific code and VEsNA Pro **effects**, is detailed in Section 6.3.

## 4 System Architecture and Workflow

The overall architecture bridges objective physical events (managed by the Java CArtAgO environment) with subjective agent reasoning (managed by the Jason/VEsNA Pro BDI engine). Figure 2 illustrates the complete lifecycle of a single turn, which operates as a continuous feedback loop.

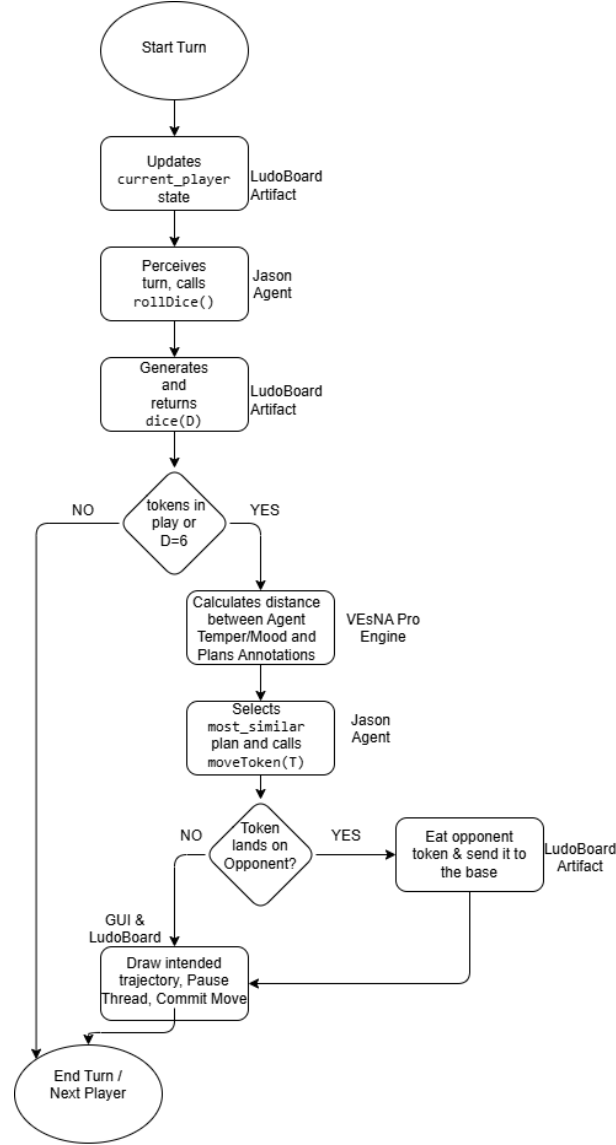


Figure 2: Complete execution flow of a single turn. The diagram illustrates the interaction between the CArtAgO LudoBoard Artifact, the Jason BDI reasoning cycle, and the VEsNA Pro emotional plan selection engine.

## 4.1 Turn Initialization and Perception

Each turn begins when the LudoBoard Artifact updates the `current_player` observable property. The corresponding Jason agent perceives this change through CArtAgO’s event notification mechanism and reacts by triggering its `+current_player(Id)` plan, which invokes the `rollDice()` artifact operation. The Artifact generates a random integer from 1 to 6 and propagates it as an update to the `dice` observable property, formally initiating the decision-making phase.

## 4.2 Legal Move Filtering and BDI Deliberation

Upon perceiving the dice value `D`, the agent enters the BDI deliberation cycle. First, **Jason evaluates the context conditions** of all plans annotated for `!choose_move(D)`. Only plans whose logical guard is provable from the current belief base are admitted as **applicable options**. For example:

- `@eat_enemy` is applicable only if `can_eat(Me, T, P, D)` holds i.e., some token can land on an occupied enemy square.
- `@exit_base` is applicable only if `D == 6` and at least one token is in the Base.
- `@advance_token` is applicable only if a token is in play and the move does not exceed position 57.

If the agent has no tokens in play and the dice result is not 6, no legal plan context is satisfied. The **fallback plan** (`@fallback_move`, with a zero-vector temper) is activated, calling `moveToken(0, _)` with no real effect, and the turn passes immediately to the next player.

Once the applicable plan set is determined, the **VEsNA Pro engine** evaluates the `temper` annotation of each applicable plan against the agent’s current full temper vector (including any live mood values for the Emotive agent). Using the `most_similar` strategy, the framework computes a vectorial similarity score and probabilistically selects the plan whose emotional and cognitive profile most closely matches the agent’s current psychological state.

## 4.3 Action Execution and Artifact Synchronization

The selected plan executes the `moveToken(T, _)` CArtAgO operation. The Artifact intercepts this call and applies the full movement logic: it computes the new logical position, checks for spatial conflicts via modular arithmetic, and if an opponent is captured, updates the opponent’s token position to `-1` and fires the corresponding `token(P, T, -1)` observable property update, **and finally** commits the final board state.

## 5 User Interface

The simulation features a custom Java Swing Graphical User Interface (GUI) integrated directly with the underlying CArtAgO Artifact. This interface serves both as a real-time game viewer and as an active **debugging and observation tool** for the multi-agent system.

### 5.1 Real-Time Playfield

The GUI renders a traditional cross-shaped Ludo board using Java2D graphics within a `BoardPanel` inner class that overrides `paintComponent(Graphics g)`. Token coordinates are **mathematically derived** from their logical positions on the cyclical array using the same formula employed by the agents, ensuring pixel-perfect consistency between the internal model and the visual output.

The board layout uses a 15×15 grid of cells, each 40 pixels wide, with the four colored corner areas representing the Base zones, the shared white perimeter encoding the 52 cyclical squares, and the four colored corridors representing the private safe zones. Tokens are rendered as filled circles with a black outline, color-coded to their respective agent personality:

- **Red** - Aggressive Agent
- **Green** - Prudent Agent
- **Yellow** - Balanced Agent
- **Blue** - Emotive Agent

A persistent **status bar** at the top of the panel displays the currently active agent's name and the result of the last dice roll, providing continuous contextual information to the observer.

### 5.2 Visual Trajectory System

A challenge in MAS simulation is exposing the internal BDI reasoning visually. To address this, a **multi-segment directional arrow system** was implemented. Before an agent finalizes a `moveToken` action, the GUI draws an arrow explicitly tracing the *exact path* the token will traverse, cell by cell, through the intermediate squares of the cyclical array.

The arrow uses two rendering layers: a thick black outer stroke for contrast and a thinner inner stroke in the token's color. A filled arrowhead polygon is computed using `Math.atan2`, ensuring correct directionality regardless of trajectory orientation. Special cases (exiting Base, entering safe zone) are handled separately.

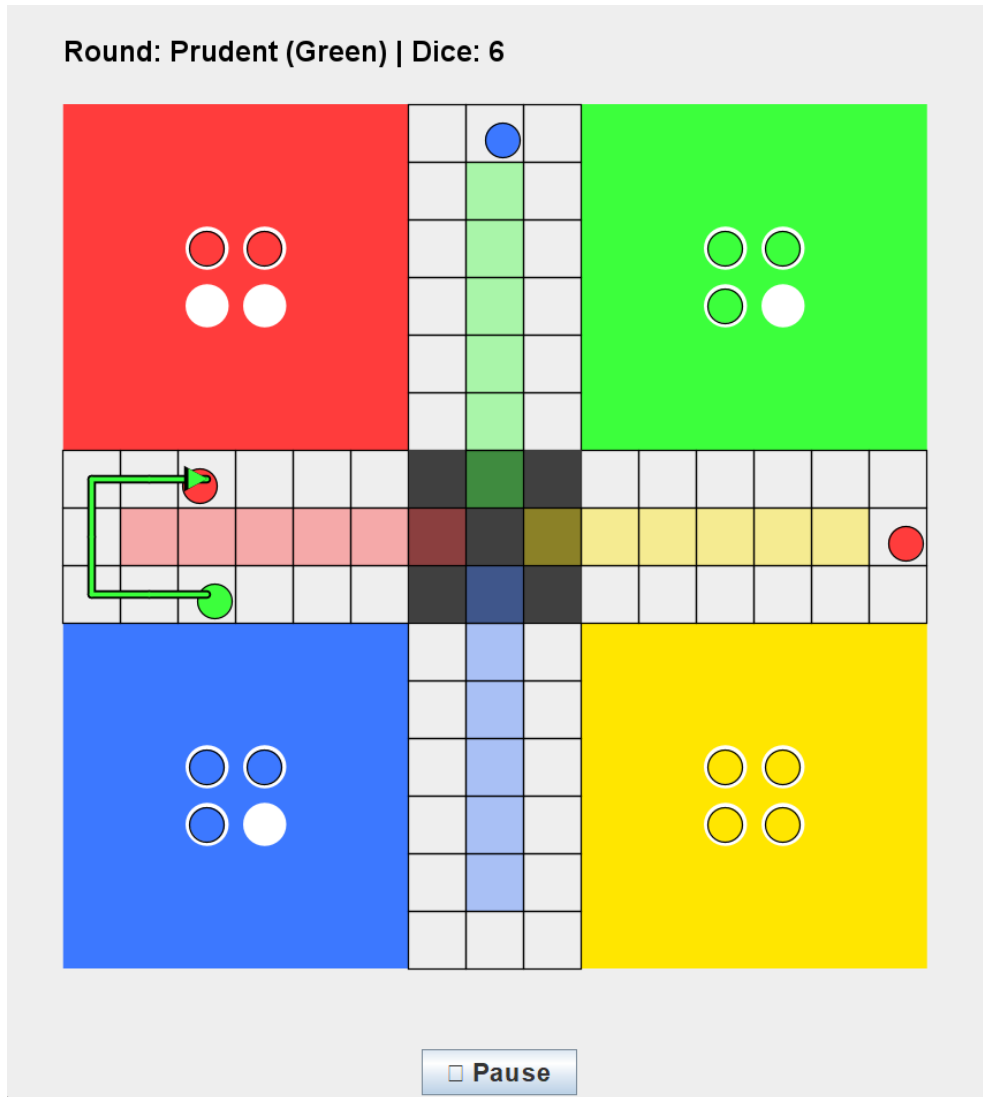


Figure 3: The Java Swing GUI during an active game turn (Blue/Emotive agent, Dice: 3). The multi-segment directional arrow traces the planned trajectory of a token moving from the safe zone entry point.

### 5.3 Pause Mechanism and Thread Safety

A **Pause/Resume** toggle button is provided in a bottom control panel. When activated, a volatile boolean `isPaused` flag is set to `true`. The Artifact's `rollDice` and `moveToken` methods each invoke `checkPause()` before any state change. This method enters a low-frequency **polling loop** (`Thread.sleep(200ms)`), yielding CPU time while awaiting resumption. This approach avoids the complexity of Java monitor locks while guaranteeing no board state mutation during the pause window, preventing deadlocks between the JaCaMo thread and the Swing EDT.

This feature allows users to freeze the simulation precisely during the trajectory visualization phase and cross-reference the visual move with the agents' BDI logging outputs in the terminal, providing real-time debugging capability.

## 6 Code Specification and Implementation

This section details the technical stack used for the MAS and highlights the core architectural implementations that drive the simulation.

### 6.1 Development Stack and Project Structure

The project is built upon the JaCaMo framework, combining three distinct technologies:

- **Jason:** The AgentSpeak interpreter used for programming the BDI agents and defining their logical rules.
- **CARTAgO:** The framework utilized to build the **LudoBoard** environment in Java, bridging the gap between object-oriented logic and Jason perception.
- **VEsNA Pro:** Integrated directly to handle static personality traits (**temper**) and dynamic emotional states. The foundational VEsNA Pro framework and base code utilized in this project were provided by the course instructors [2].

The codebase follows the JaCaMo directory convention: The `src/env` directory contains the Java CARTAgO artifacts and GUI components, while the `src/agt` directory contains the `.asl` source files for the BDI agents. The entire system is orchestrated by the main `.jcm` configuration file.

### 6.2 Spatial Collision Detection (`can_eat`)

The `can_eat` rule is the mathematical core shared by the Aggressive, Balanced, and Emotive agents. It determines whether a specific token move would land on a square occupied by an enemy:

```

1 can_eat(Me, T, P, D) :-
2     token(Me, T, P) & P >= 0 & (P + D) <= 50 &
3     token(Enemy, _, EnemyP) & Enemy \== Me & EnemyP >= 0 &
4     EnemyP <= 50 &
    ((Me * 13 + P + D) mod 52) == ((Enemy * 13 + EnemyP)
    mod 52).

```

The rule verifies three conditions simultaneously: the source token is currently in play ( $P \geq 0$ ); the move stays within the shared perimeter ( $P + D \leq 50$ ); and the destination's absolute position, computed via the modulo formula, coincides with an enemy's absolute position. This single rule replaces what would otherwise require iterating over every possible board combination explicitly.

### 6.3 Emotional Annotations and Vector Alignment

A critical design challenge in integrating VEsNA Pro was ensuring that the Cosine Similarity calculation prioritizes dynamic moods over static personality traits. To achieve this, the static traits (**aggression**, **safety**, **advance**) across all of the Emotive agent's plans were mathematically locked to a neutral baseline of 0.5. By eliminating the variance in these three dimensions, the vector distance calculated by the engine is theoretically forced to depend entirely on the mutable [mood] values (**anger** and **joy**).

The intended transition from a calm state to a retaliatory state and the subsequent catharsis is structured through plan annotations. The following snippet illustrates the design of the `@vendetta` plan:

```

// Retaliatory action requiring high anger while maintaining the neutral baseline
@vendetta[
    temper([aggression(0.5), safety(0.5), advance(0.5), anger(0.9), joy(0.1)]),
    effects([anger(0.0), joy(1.0)])
]
+!choose_move(D) : my_id(Me) & can_eat(Me, T, P, D) <-
    .print(" Emotive Move (Catharsis): Token ", T, " captured an enemy!");
    moveToken(T, _).

```

From an architectural standpoint, this strict alignment was designed to guarantee that when the agent experiences trauma, the **anger(0.9)** annotation would naturally dominate the similarity score, forcing the selection of retaliatory actions. Furthermore, the **effects** vector was explicitly configured to use absolute boundary values (**anger(0.0)**, **joy(1.0)**) to ensure a clean mood reset upon successful retaliation.

However, as detailed in Section 7.2, empirical testing revealed that while this vector alignment is mathematically sound, the actual execution of the emotional loop is hindered by framework-level limitations regarding mood state persistence.

## 6.4 Environment and Concurrency Management

The `LudoBoard.java` artifact acts as the absolute source of truth. As detailed extensively in Section 5.3, the `moveToken` operation prevents race conditions between the JaCaMo reasoning cycle and the Java Swing EDT via a polling loop, successfully avoiding monitor lock complexities and ensuring thread safety.

## 6.5 System Configuration (.jcm)

The `.jcm` file declares the full MAS, including agent classes, temper values, WebSocket addresses, and initial goals. The Emotive agent's entry demonstrates the `[mood]` tag syntax:

```
agent emotive : emotive_agent.asl {
    ag-class:  vesna.VesnaAgent
    temper:    temper( aggression(0.5), safety(0.5), advance(0.5),
                      anger(0.1)[mood], joy(0.8)[mood] )
    strategy:  most_similar
    address:   localhost
    port:      9083
    goals:     start
}
```

The `[mood]` tag is critical: without it, VEsNA Pro treats **anger** and **joy** as immutable personality traits and silently ignores all **effects** at runtime, disabling the emotional feedback system entirely.

## 6.6 Execution Instructions

To run the simulation, ensure that a compatible Java Development Kit (JDK) and Python 3 are installed. The execution requires two sequential steps:

1. **Start the Dummy Server:** Open a terminal, navigate to the server directory, and execute the Python WebSocket script to initialize the body endpoints for the agents:

```
python dummy_server.py
```

2. **Launch the MAS:** Once the Python server is actively listening, open a new terminal in the root project directory and use Gradle to build and run the multi-agent system:

```
gradle run
```

Upon successful execution, the Java Swing GUI will automatically initialize, the agents will establish their WebSocket connections to the dummy server, and the autonomous Ludo simulation will begin. The simulation can then be controlled via the Pause/Resume button in the interface.

## 6.7 Use of Artificial Intelligence and LLMs

LLMs and Generative AI were utilized as auxiliary tools during the development of this project for the following specific tasks:

- **Code Generation and Setup:** To automate the writing of repetitive structural components and to assist in the initial setup of the project structure.
- **GUI syntax:** Providing guidance on Java Swing API usage and layout management.
- **Language and Documentation:** To perform grammatical, stylistic, and formatting checks throughout this academic report.

## 7 Results and Observations

This section presents and analyses the behavioral patterns observed across multiple simulation runs, drawing directly from system log outputs. The goal is to verify whether the VEsNA Pro temper configuration produces meaningfully distinct agent behaviors, and whether the emotional state machine of the Emotive agent functions as designed.

### 7.1 Observed Agent Behavioral Patterns

All four agents demonstrated recognizably distinct behavioral profiles consistent with their configured `temper` vectors.

**Aggressive Agent.** The Aggressive agent’s advanced hunting logic implemented via `.findall` and `.min` on circular modular distances was consistently active throughout observed games. The following log sequence is representative:

```
1 [aggressive] I rolled a 4. Looking for the best move...
2 [aggressive] Aggressive Move : Move the token 0 to get
   closer to the prey.
3 [aggressive] I rolled a 5. Looking for the best move...
4 [aggressive] Aggressive Move : Use 5 to EAT an enemy with
   token 0!
5 EATEN P0 has eaten a token from P3
```

This sequence confirms that `@hunt_pre`y correctly positioned the token at the optimal chasing distance, and the subsequent `@eat_enemy` plan was triggered on the following turn as intended. In one observed full game, the Aggressive agent performed the most captures of any agent, validating the effectiveness of its targeting algorithm.

**Prudent Agent.** The Prudent agent demonstrated a highly risk-averse, "one-at-a-time" strategy. Because the `@exit_base` plan has the lowest priority (`safety = 0.2`), the agent strongly prefers advancing an already exposed token rather than deploying new ones. Once a token is in play, the agent consistently focuses all its dice rolls on moving that specific token toward the safe corridor:

```

1 [prudent] I rolled a 6. Looking for the best move...
2 [prudent] Prudent Move: I move the token 0 to be safe from
   the enemies.

```

The agent acts to minimize its footprint on the shared perimeter, attempting to secure one token in the safe zone before bringing a new target onto the board. When the dice roll allows a token to step into the final private corridor, the agent reliably prioritizes that absolute safety over any other available action.

**Balanced Agent.** The Balanced agent's army strategy is the main concept. The `active_tokens` counter correctly triggered the urgent deployment plan when the agent had fewer than 2 tokens in play:

```

1 [balanced] I rolled a 6. Looking for the best move...
2 [balanced] Balanced Move (Army): I have only 0 tokens out!
   I use the 6 to deploy the token 3!
3 ...
4 [balanced] I rolled a 6. Looking for the best move...
5 [balanced] Balanced Move (Army): I have only 1 tokens out!
   I use the 6 to deploy the token 2!

```

Once two tokens were active, the agent transitioned smoothly to the standard advancement strategy (`@advance_token`), demonstrating good dynamic priority management. This continuous re-evaluation proved highly adaptive during game-play: if opponents captured its tokens, the agent's internal active token count dropped accordingly. This immediately caused the agent to shift its priority back to urgent deployment upon its next roll of 6, automatically attempting to rebuild its presence on the board rather than stubbornly advancing a single surviving token.

## 7.2 Attempted Emotional Transitions (Emotive Agent)

The intended design for the Emotive agent was to create a closed feedback loop where a token capture triggers an angry retaliatory state, which is eventually resolved through catharsis. However, empirical observation of the simulation logs revealed a critical failure in the emotional state machine's execution. The following log excerpt captures the anomaly:

```

1   EATEN P2 has eaten a token from P3
2   [emotive_player] Emotive Event: Token 0 was captured! Anger
    level spiked to maximum.
3   [emotive_player] I rolled a 1. Looking for the best move...
4   [emotive_player] I pass this turn because I have no valid
    moves.
5   ...
6   [emotive_player] I rolled a 6. Looking for the best move...
7   [emotive_player] Emotive Move (Calm): Peacefully deploying
    token 0 from the base.

```

This sequence demonstrates a clear disconnect between the BDI logical layer and the VEsNA Pro emotional engine. The Jason interpreter correctly intercepts the capture event via the `in_play(T)` working memory and successfully fires the internal trauma trigger (printing the spike in anger). However, the VEsNA Pro framework fails to persist the `effects([anger(1.0), joy(0.0)])` mutation across turn boundaries.

When the agent is called to deliberate on its next valid move (e.g., upon rolling a 6), the internal mood vector has already reset to its initial state. The `most_similar` strategy evaluates the applicable plans using the original calm baseline (`joy = 0.8`), completely ignoring the trauma. Consequently, the agent incorrectly selects the `@exit_base_calm` plan, rendering the "Angry" state and the subsequent "`@vendetta`" catharsis practically unreachable during live gameplay.

### 7.3 Limitations

Several limitations were identified during the evaluation phase:

- **No Inter-Agent Communication:** Agents have no mechanism to share information or coordinate strategies. All reasoning is performed independently based on the shared observable properties of the artifact. This is a fundamental constraint of the current JaCaMo configuration and eliminates the possibility of coalition or deception strategies.
- **VEsNA Pro Mood State Persistence:** As highlighted in Section 7.2, a systemic limitation was observed within the VEsNA engine's state management. When `[mood] effects` are triggered dynamically by reactive sub-goals (such as intercepting a capture event outside the standard `!choose_move` deliberation cycle), the framework fails to persist the updated mood vector for subsequent turns. This prevents the execution of complex emotional state machines, as the agent suffers from "emotional amnesia" between turns.

## 8 Conclusions and Future Improvements

This project demonstrates the design and implementation of a competitive Multi-Agent System that successfully models distinct tactical profiles through VEsNA Pro temper vectors, while exposing important limitations in the framework’s emotional state persistence mechanisms.

The attempt to model dynamic emotional states (trauma, anger, and catharsis) via the VEsNA Pro framework provided valuable architectural insights. While the vector-based **temper** alignment was mathematically sound, the simulation exposed critical limitations in the framework’s ability to persist emotional mutations (**effects**) across turn boundaries. This ”emotional amnesia” highlights a crucial area for future development in affective computing frameworks: the need for robust state persistence when emotions are triggered asynchronously by environmental trauma rather than standard deliberate actions.

### 8.1 Future Improvements

While the current simulation effectively proves the viability of combining cognitive and emotional AI, future iterations of this project could significantly expand the emotional complexity of the agents:

- **Full OCC Emotional Model:** The current implementation focuses primarily on Anger and Joy. Integrating the broader OCC (Ortony, Clore, Collins) emotional model would allow agents to experience a wider spectrum of states. For instance, an agent could compute ”Fear” when a highly Aggressive opponent is spatially approaching from behind, or ”Relief/Satisfaction” upon successfully moving a token into the absolute safety of the inner zone.
- **Human-in-the-Loop (Hybrid Mode):** The current environment is a closed-loop AI simulation. Introducing a hybrid mode that allows a human player to control one set of tokens against three VEsNA-driven AI profiles would be a highly valuable addition. This would provide empirical data on human-AI tactical interactions, allowing researchers to measure if human players can recognize, predict, or be manipulated by the emotional states of their virtual opponents.

## References

- [1] Ludo game. <https://en.wikipedia.org/wiki/Ludo>
- [2] Andrea Gatti, A.F.: Vesna-toolkit: vesna-pro. <https://github.com/VEsNA-ToolKit/vesna-pro>

- [3] Nettikadan, K.: Ludo\_mas. [https://github.com/Kevinnetti/Ludo\\_Mas](https://github.com/Kevinnetti/Ludo_Mas) (2026)
- [4] Oracle Corporation: Creating a gui with jfc/swing - the java tutorials. <https://docs.oracle.com/javase/tutorial/uiswing/>